BTS Services Informatiques aux Organisations
Fiche hebdomadaire de suivi de stage

PARCOURS SISR ☐	**PARCOURS SLAM** ☑

Semaine n° 4 du **26 janvier** au **30 janvier** 2026

Nom de l'étudiant : PORTOLLEAU Anaïs

Nom de l'organisation d'accueil : DELIA TECHNOLOGIES

Nom, adresse e-mail et/ou téléphone du tuteur au sein de l'organisation :

Tuteur : Julien Relandeau
Contacter Pauline Francheteau : 06 14 95 06 49

Nom du tuteur école : Ouafâa Laouija / Nicolas Bourgeois

Rappel du contexte *(activité de l'organisation, nom du service d'accueil et rôle, projet éventuel, …)* :

DELIA TECHNOLOGIES est une ESN (Entreprise de Services du Numérique) fondée en 2019 et basée à Nantes. Elle accompagne ses clients dans la conception, le développement et la maintenance de solutions numériques innovantes, en apportant une expertise technique adaptée à leurs besoins. Une seconde ESN a également été créée en 2021 à Rennes.

À la fin de l'année 2025, une nouvelle version majeure de Node.js ainsi qu'une évolution de l'API Pennylane ont été déployées. Cette évolution technologique a rendu nécessaire la mise à jour de plusieurs projets afin de garantir leur compatibilité, leur sécurité et leur bon fonctionnement dans le cadre de leur exploitation en 2026.

Dans ce contexte, j'ai été amenée à intervenir sur une application existante nommée **Lucas 2 Penny**. Il s'agit d'une application métier permettant notamment la gestion des factures, des données internes de l'entreprise ainsi que d'autres fonctionnalités essentielles. Le projet n'ayant pas été migré depuis une mise à jour datant d'environ juin 2025, une migration vers la nouvelle version était indispensable.

Cette application repose sur l'utilisation de **deux API externes nommées Lucca et Pennylane**.

Mon rôle a consisté à contribuer à cette évolution technique en adaptant le code existant, en identifiant les incompatibilités liées à la montée de la version 2 de l'application, ainsi qu'aux évolutions de l'une des API qui est Pennylane.

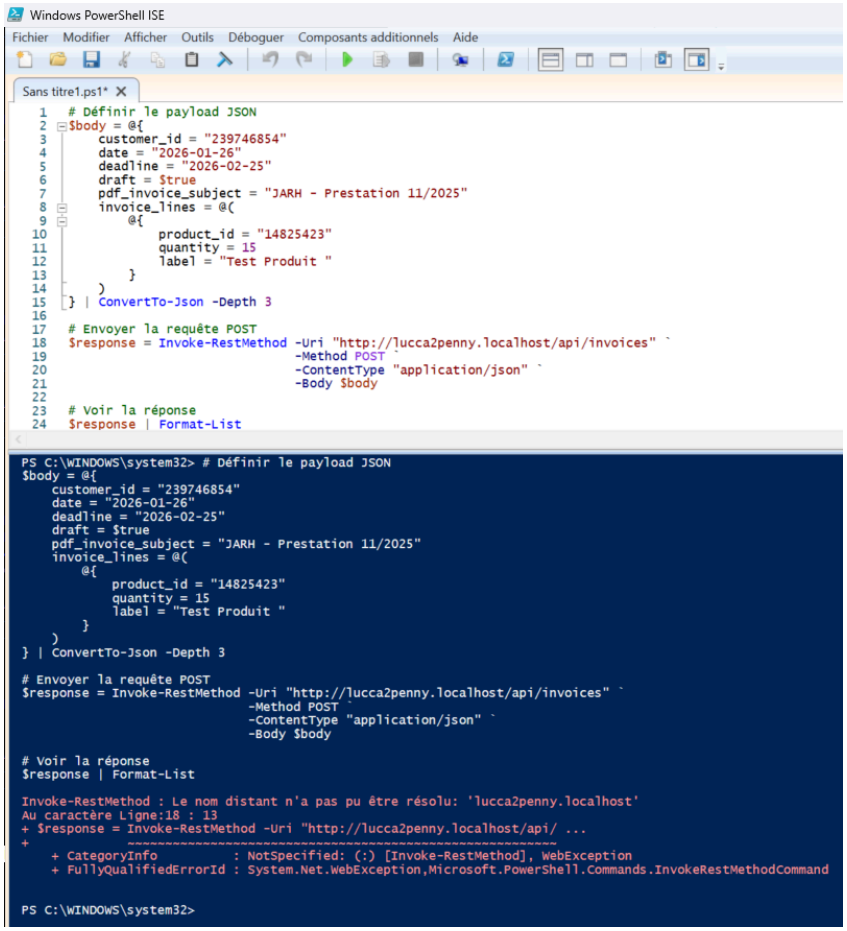
Compte rendu du travail effectué *(liste des tâches avec leur description)* :

1. Vérification du travail de la semaine précédente

- Contrôle de l'affichage des transactions et des factures dans les tableaux, correspondance avec les données de Pennylane

2. Tests initiaux de génération des factures

- Tests via PowerShell et Postman : la facture est créée mais ne se met pas à jour dans la base après actualisation.


Windows PowerShell ISE
Fichier Modifier Afficher Outils Déboguer Composants additionnels Aide
Sans titre1.ps1*
# Définir le payload JSON
$body = @{
customer_id = "239746854"
date = "2026-01-26"
deadline = "2026-02-25"
draft = $true
pdf_invoice_subject = "JARH - Prestation 11/2025"
invoice_lines = @(
@{
product_id = "14825423"
quantity = 15
label = "Test Produit "
}
)
} | ConvertTo-Json -Depth 3
Envoyer la requête POST
$response = Invoke-RestMethod -Uri "http://lucca2penny.localhost/api/invoices" `
-Method POST `
-ContentType "application/json" `
-Body $body
Voir la réponse
$response | Format-List
PS C:\WINDOWS\system32> # Définir le payload JSON
$body = @{
customer_id = "239746854"
date = "2026-01-26"
deadline = "2026-02-25"
draft = $true
pdf_invoice_subject = "JARH - Prestation 11/2025"
invoice_lines = @(
@{
product_id = "14825423"
quantity = 15
label = "Test Produit "
}
)
} | ConvertTo-Json -Depth 3
Envoyer la requête POST
$response = Invoke-RestMethod -Uri "http://lucca2penny.localhost/api/invoices" `
-Method POST `
-ContentType "application/json" `
-Body $body
Voir la réponse
$response | Format-List
Invoke-RestMethod : Le nom distant n'a pas pu être résolu: 'lucca2penny.localhost'
Au caractère Ligne:18 : 13
+ $response = Invoke-RestMethod -Uri "http://lucca2penny.localhost/api/ ...
+ ~~~

Accounting / api / external / v2 / supplier_invoices / **List supplier invoices**

POST https://app.pennylane.com/api/external/v2/customer_invoices

Docs Params Authorization • Headers (13) Body • Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1  {
2    "customer_id": "239746854",
3    "date": "2026-01-26",
4    "deadline": "2026-02-25",
5    "draft": true,
6    "pdf_invoice_subject": "JARH - Prestation 11/2025",
7    "invoice_lines": [
8      {
9        "product_id": "14825423",
10       "quantity": 15,
11       "label": "Test Produit "
12     }
13   ]
14 }
15
```

Body Cookies (2) Headers (21) Test Results

{} JSON Preview Visualize

```
1  {
2    "id": 4012248789,
3    "label": "Facture TEST (label généré)",
4    "invoice_number": "",
5    "currency": "EUR",
6    "amount": "90.0",
7    "currency_amount": "90.0",
8    "currency_amount_before_tax": "75.0",
9    "tax": "15.0",
10   "currency_tax": "15.0",
11   "remaining_amount_with_tax": null,
12   "remaining_amount_without_tax": null,
13   "draft": true,
14   "language": "fr_FR",
15   "paid": false,
16   "status": "draft",
17   "filename": null,
```

J'ai fait un test dans postman pour vérifier que le token marchait toujours après un message d'erreur qui m'indiquait que le token était invalide

- Vérification dans la base de données locale, on peut voir qu'au départ, la table était vide.

`SELECT * FROM "invoices" LIMIT 50` (0.006 s) Modifier

Aucun résultat.

Importer

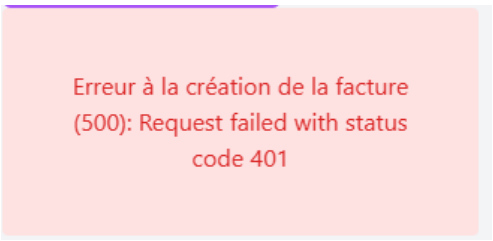
- Génération de factures en changeant la méthode en entrant des données aléatoires et ont été ensuite insérées dans la base, mais certaines lignes sont manquantes → 1 seul résultat

`SELECT * FROM "invoices" LIMIT 50` (0.001 s) Modifier

Modification	id	luccaProjectId	month	code	pennylaneId	
modifier	1	CLDEVO01-PRJANCDEVODEJS01	2026-01	CLDEVO01-PRJANCDEVODEJS01-106-2026-01	idAZERTYIOUZHOEZNDFOIZED	202

Résultat entier: 1 ligne | Modification: Enregistrer | Sélectionnée(s) (0): Modifier Cloner Effacer | Exporter (1)

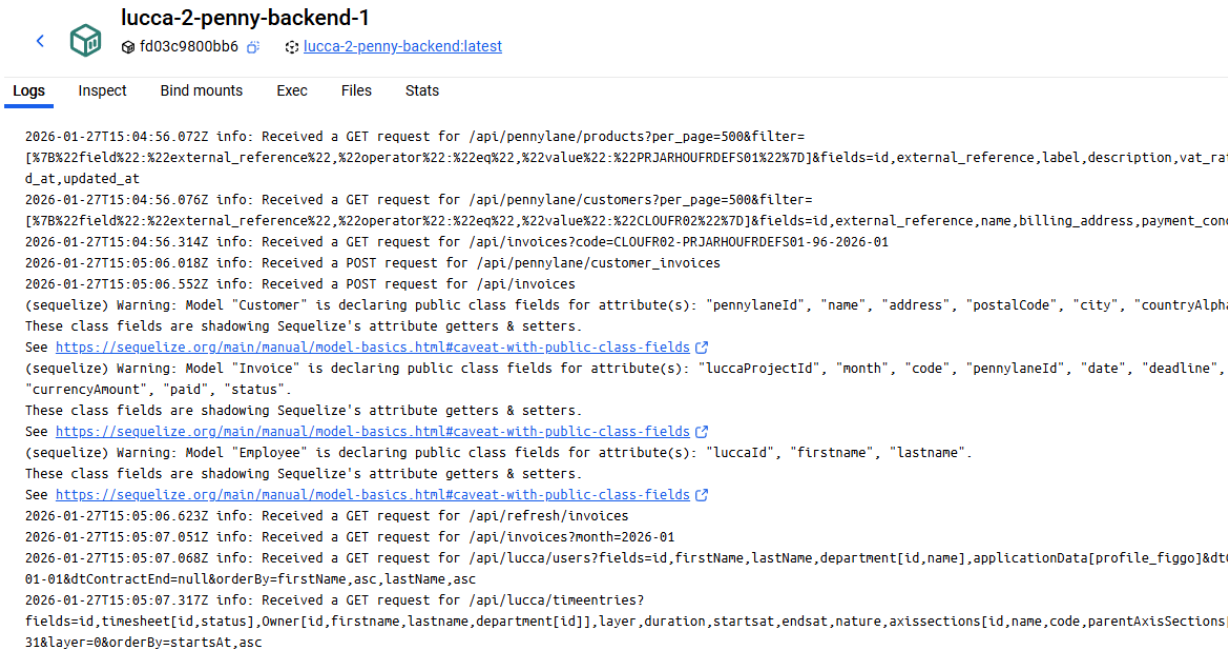
- J'ai rencontré durant mes tests, plusieurs erreurs 500, côté front (sur l'application et sur la console du navigateur) empêchant la validation complète du processus de création des factures.



Puis dans la console du navigateur, on peut voir qu'il a bien récupéré les données qu'on souhaite avoir (4 premières lignes), mais on rencontre à la fin une erreur 500 qui serait la cause des soucis (5ème ligne ci-dessous) :

customers?per_page=500&filter=[%7B%22field%22:%22...	200	lucca2penny.localhost	xhr	api.ts	8,6 ko	371 m
products?per_page=500&filter=[%7B%22field%22:%22e...	200	lucca2penny.localhost	xhr	api.ts	8,4 ko	250 m
invoices?code=CLOUFR02-PRJARHOUFRDEFS01-96-202...	200	lucca2penny.localhost	xhr	api.ts	0,3 ko	23 m
customer_invoices	201	lucca2penny.localhost	xhr	api.ts	10,1 ko	1,34
invoices	500	lucca2penny.localhost	xhr	api.ts	1,7 ko	231 m

- Puis j'ai pu effectuer une analyse des logs Docker : on peut voir que les factures sont créées mais sans lignes associées (lineItems: []).



3. Modification et correction du code

- Pour faire fonctionner la fonctionnalité de création des factures, j'ai fait la mise à jour des variables et méthodes pour correspondre aux changements de l'API Pennylane (de la v1 vers la v2) en ajustant le code côté back et côté front selon la documentation officielle de pennylane (Lien)

- Résolution partielle du problème du brouillon en supprimant certains rafraîchissements inutiles tout d'abord en supprimant certains refresh()), mais cette suppression doit être temporaire car il faut éviter de faire F5.

En continuant à faire des logs j'ai trouvé un ce log qui m'a beaucoup aidé car cela m'a permis de voir qu'il ya avait une facture qui se crée mais ne s'enregistrait pas :

Containers / lucca-2-penny-backend-1

lucca-2-penny-backend-1

fd03c9800bb6 lucca-2-penny-backend:latest

Logs Inspect Bind mounts Exec Files Stats

```
(sequelize) Warning: Model "Invoice" is declaring public class fields for attribute(s): "luccaProjectId", "month", "code", "pennylan
"currencyAmount", "paid", "status".
These class fields are shadowing Sequelize's attribute getters & setters.
See https://sequelize.org/main/manual/model-basics.html#caveat-with-public-class-fields
[CREATE INVOICE] Employé trouvé/créé: { id: 1, luccaId: 96, firstname: 'Jacques', lastname: 'ARHAN' }
(sequelize) Warning: Model "Employee" is declaring public class fields for attribute(s): "luccaId", "firstname", "lastname".
These class fields are shadowing Sequelize's attribute getters & setters.
See https://sequelize.org/main/manual/model-basics.html#caveat-with-public-class-fields
[CREATE INVOICE] Facture sauvegardée avec ID: 3
[CREATE INVOICE] Pas de lignes à créer pour cette facture
[CREATE INVOICE] Facture finale récupérée: {
  id: 3,
  luccaProjectId: 'CLOUFR02-PRJARHOUFRDEFS01',
  month: '2026-01',
  code: 'CLOUFR02-PRJARHOUFRDEFS01-96-2026-01',
  pennylaneId: '4035803670',
  date: 2026-01-27T00:00:00.000Z,
```

Ce qui m'a permis de voir dans quelle méthode cela n'allait pas et de la modifier en conséquence.

4. Gestion du logo euro (symbol €)

- Après mes modifications, le logo euro ne passait en orange qu'après un F5. Après plusieurs jours d'analyse des logs, mon tuteur m'a informé qu'il n'était pas au courant que ce changement devait fonctionner via un bouton « Mise à jour ». J'ai donc adapté le fonctionnement afin que le passage du symbole d'orange à vert se fasse via le bouton « Mettre à jour ».

- Modification de la méthode updateCraAndInvoices :

```
44    const updateCraAndInvoices = async () => {
45+       // Mémoriser les projets actuellement en brouillon
46+       const currentDrafts = new Set<string>();
47        displayedItems.forEach(item =>
48            item.projects.forEach(p => {
49+               if (p.status === 'draft') currentDrafts.add(p.id);
50            })
51        );
52        setDraftProjectIds(currentDrafts);

53+       // Rafraîchir les factures depuis Pennylane
54        const success = await refreshInvoicesFromPennylane();
55+       if (!success) return;
56+       // Re-fetch des données Lucca
57        const { data } = await refetch();
58+       if (!data) return;
59+       // Filtrer selon la recherche et les départements
60        const filteredItems = data.filter(item =>
61            (currentDepartments.length === 0 || (item.department && currentDepartments.includes(item.department.id))) &&
62            (item.firstName.toLowerCase().includes(searchQuery.toLowerCase()) ||
63            item.lastName.toLowerCase().includes(searchQuery.toLowerCase()) ||
64+           item.projects.some(p => p.name.toLowerCase().includes(searchQuery.toLowerCase())))
65        );
66+       // Fusionner les statuts
67+       const updatedItems: ViewByDayItem[] = filteredItems.map(item => ({
68+           ...item,
69+           projects: item.projects.map(p => {
70+               const isDraft = currentDrafts.has(p.id); // projet était en brouillon avant update
71+               const isFactured = !!p.factured || !!p.fileUrl; // projet déjà facturé
72+
73+               return {
74+                   ...p,
75+                   // Si c'était un brouillon → garder draft
76+                   // Sinon si facturé → validated
77+                   // Sinon garder le statut actuel (A facturer)
78+                   status: isDraft ? 'draft' : isFactured ? 'validated' : p.status,
79+                   factured: isFactured, // boolean strict
80+               };
81+           })
82+       }));
83+       // Mettre à jour l'affichage
84+       setDisplayedItems(updatedItems);
85+   };
86+
```

Anais Portolleau (17 hours ago) Ln 14, Col 1 Spaces: 4 UTF-8 TypeScript JSX

J'ai modifié la fonction updateCraAndInvoices pour mémoriser les projets en brouillon et fusionner correctement les statuts après la mise à jour, garantissant que les factures passent du statut brouillon à valider ou restent à facturer selon le cas

- Modification des méthodes et affichage du texte « À facturer » après clic sur le bouton.

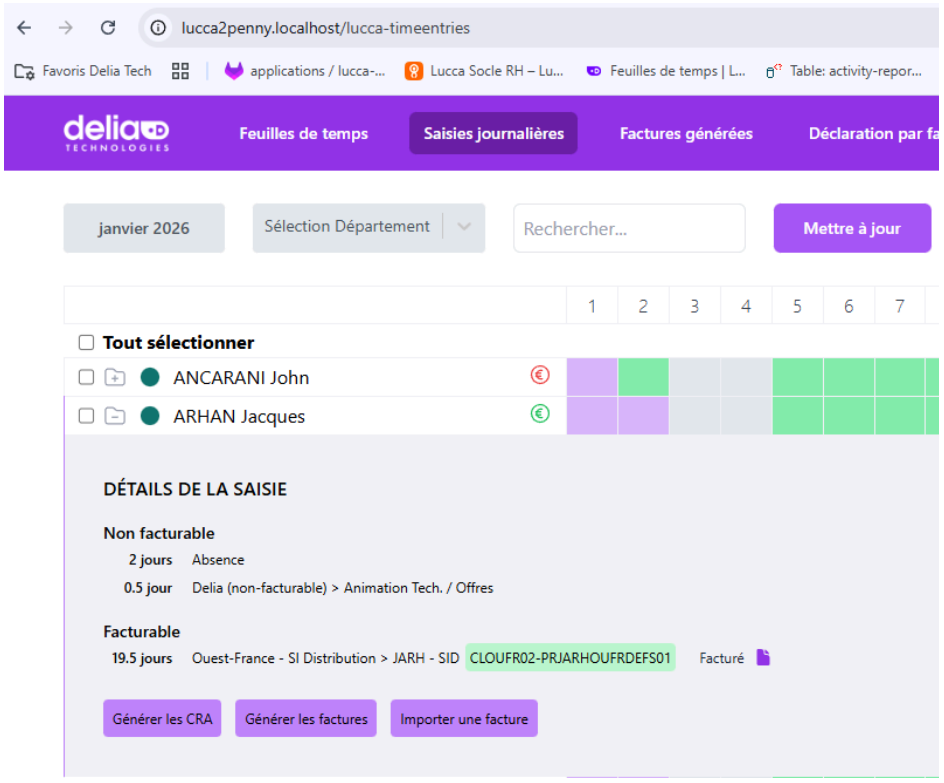
- Validation finale : générer une facture sans la mettre en brouillon puis cliquer sur **Mettre à jour**, le symbole passe au **vert** et le texte s'affiche correctement.

5. Validation finale

- Le passage du symbole euro du statut orange au vert fonctionne désormais dans tous les cas testés.

- Les modifications ont été testées et validées sur différentes factures et cas d'utilisation.

Quand je teste, on veut générer pour Jacques sa facture, on appuie sur Générer les factures ça marche que dans le cas où ce n'est pas en brouillon que du rouge au vert :

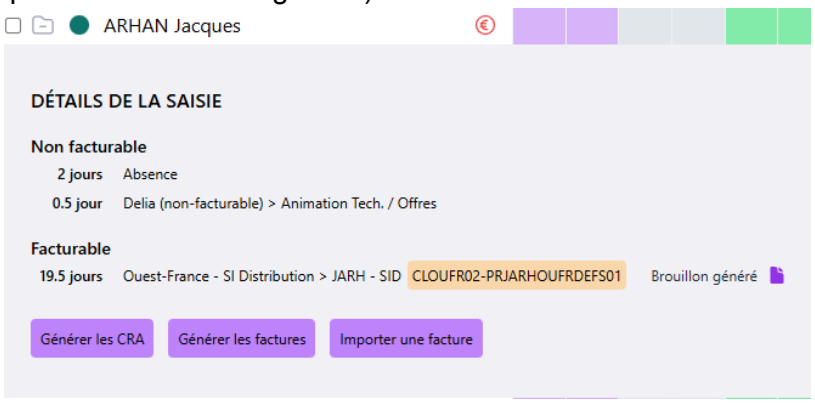


Pour régler le soucis pour les brouillon en partie dans un premier temps j'ai enlevé certains refresh() :

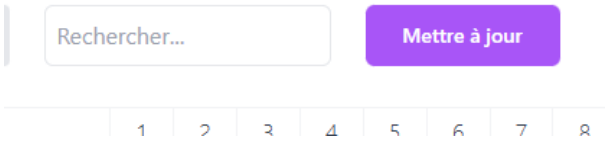
```
const closeModal = () => { // Fonction de fermeture de la modal de facturation
    setFacturationOpened(false);
   // refresh();
};

const onModalClose = async () => {
    setImportInvoiceOpened(false);
    refresh();
};
```

Pour le brouillon par contre j'ai réussi à régler mon problème en enlevant un des refresh (on peut voir qu'un brouillon a été généré):



Puis à faire passer le logo Euro en orange mais par contre il s'affiche que après un F5 :



J'ai réussi par la suite en modifiant le code et notamment createinvocie:

```
const createInvoice = async () => {
    setMessage('');

    if (!displayedInvoice || !selectedUser) return;

    try {
        // Vérifier si une facture existe déjà
        const existingInvoices: Invoice[] = await fetchInvoicesByPennyId(displayedInvoice.id);
        const luccaProjectId = `${displayedInvoice.customer?.external_reference}-${selectedUser.id}-${selectedMonth}`;
        const codeProject = `${luccaProjectId}-${selectedUser.id}-${selectedMonth}`;

        if (existingInvoices.length > 0) {
            setIsSuccess(false);
            setMessage(
                `Une facture avec un identifiant Pennylane similaire (${displayedInvoice.id}) existe déjà`
            );
            return;
        }

        // Tout est OK, on peut rattacher la facture
        setIsSuccess(true);
        setMessage(`La facture a été rattachée, code (${codeProject})`);

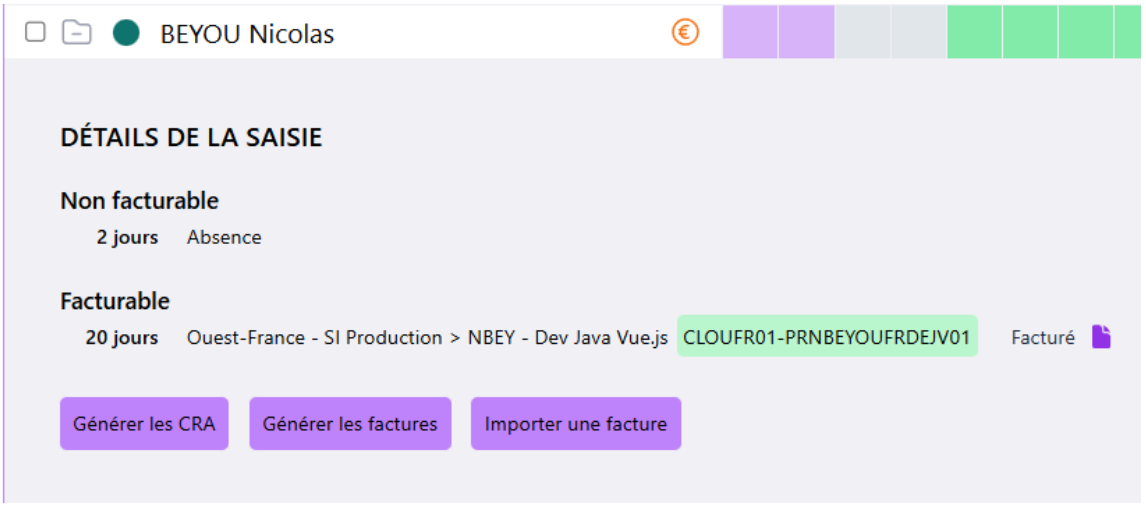
        // Envoi vers le backend + création des lignes
        await sendInvoiceInfosToBackend(displayedInvoice, refresh);

        // Invalider les queries react-query pour rafraîchir les vues
        await queryClient.invalidateQueries('invoices');
        await queryClient.invalidateQueries('projects');
        await queryClient.invalidateQueries('cra');

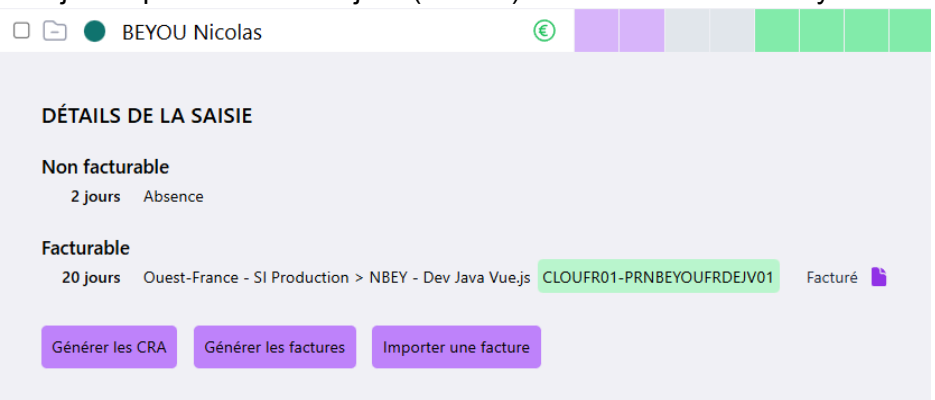
        // Rafraîchir la vue locale
        refresh();
    } catch (error) {
        console.error('Erreur lors de la création de la facture :', error);
        setIsSuccess(false);
        setMessage('Une erreur est survenue lors du rattachement de la facture.');
    }
};
```

Donc elle passait bien passé orange quand je cliquais sur mettre à jour mais pour passer du orange au vert cela ne fonctionnait pas donc j'ai du corriger ce bug et après avoir galéré pendant des jours ça fonctionnait enfin grâce aux différentes méthodes modifiés comme et à changer les informations comme le texte qui devait apparaître " À facturer" après le clic sur le bouton Mettre à jour :

Puis on regènère une facture mais cette fois-ci on ne met pas en brouillon :



Puis j'ai cliqué sur Mettre à jour (bouton) et cela remet bien le symbole euro en vert :



J'ai identifié et corrigé un bug qui empêchait le symbole euro de passer du statut orange au statut vert lors du clic sur le bouton **« Mettre à jour »**. J'ai ajusté les différentes méthodes concernées ainsi que les informations affichées afin que le texte correct, **« À facturer »**, apparaisse après la mise à jour. Après plusieurs jours de travail et de tests, le passage du symbole au vert fonctionne désormais correctement. La correction a été validée en générant une facture sans la mettre en brouillon, puis en cliquant sur **« Mettre à jour »**, ce qui met à jour le statut et le texte comme attendu.

6. Rattachement des factures Pennylane à un client

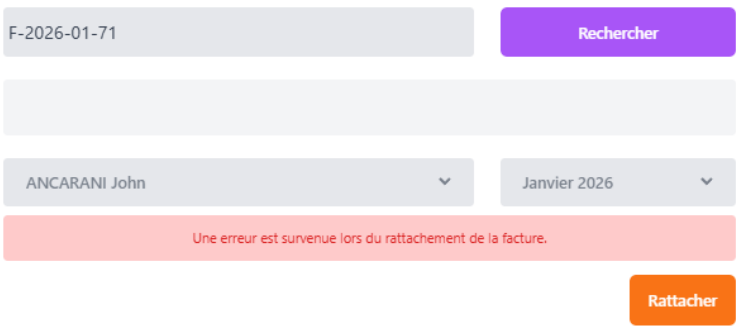
- **Changement de la méthode pour rechercher par nombre :**

```typescript
/**
 * Recherche par invoice_number
 */
export const fetchPennylaneInvoiceByIdNumber = async (
    invoiceNumber: string
): Promise<IPennylaneInvoice | null> => {
    // Création du filtre pour rechercher par invoice_number
    const filters: IPennylaneFilter[] = [
        {
            field: 'invoice_number',
            operator: PennylaneApiOperator.equalTo,
            value: invoiceNumber,
        },
    ];

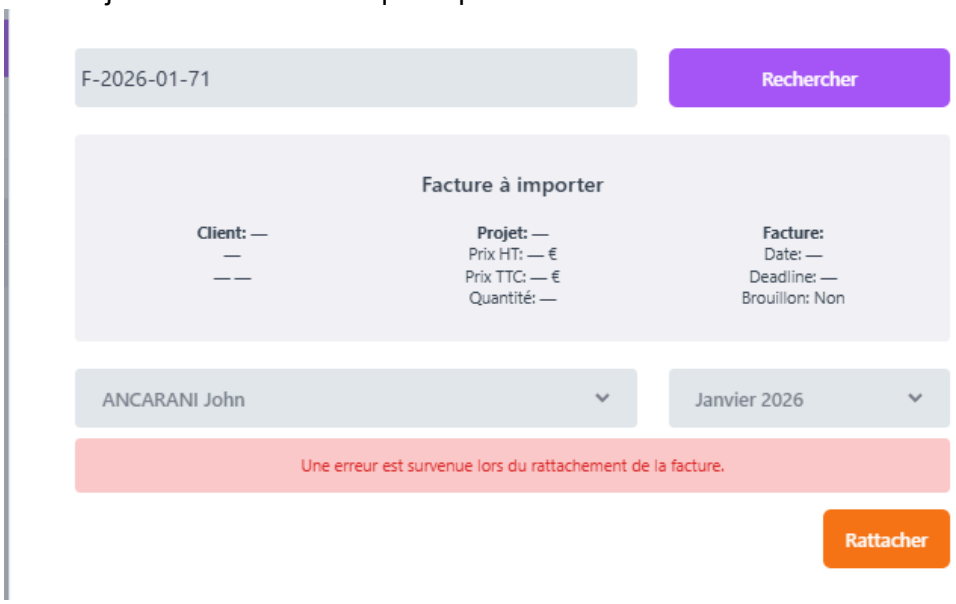
    // Appel à l'API Pennylane (ou mock)
    const pennylaneInvoices: IPennylaneInvoice[] = await fetchPennylaneInvoices(
        null, // cursor initial
        filters
    );

    // On retourne le premier résultat s'il existe
    return pennylaneInvoices.length > 0 ? pennylaneInvoices[0] : null;
};
```

Après test on peut voir que j'ai eu une erreur lors du rattachement de la facture :



- Ensuite j'ai modifié la Modal qui va permettre de rattacher la facture à un client :



Voici la partie que j'ai modifié :

```
{displayedInvoice && (
<div className="flex flex-col py-6 px-6 mt-5 space-y-4 bg-gray-100 rounded">
    <h2 className="text-md font-semibold text-center">Facture à importer</h2>
    <div className="flex space-x-6 mt-2">
        {/* Informations client */}
        <div className="w-1/3 text-xs">
            <strong>Client:</strong> {displayedInvoice.customer?.name || '—'}<br />
            {displayedInvoice.customer?.billing_address?.address || '—'}<br />
            {displayedInvoice.customer?.billing_address?.postal_code || '—'}{' '}
            {displayedInvoice.customer?.billing_address?.city || '—'}
        </div>

        {/* Informations projet / ligne */}
        <div className="w-1/3 text-xs">
            {(() => {
                const invoiceLines = Array.isArray(displayedInvoice.invoice_lines) ? displayedInvoice.invoice_lines : [];
                const firstLine = invoiceLines[0];
                return (
                    <>
                        <strong>Projet:</strong> {firstLine?.label || '—'}<br />
                        Prix HT: {firstLine?.currency_price_before_tax ?? '—'} €<br />
                        Prix TTC: {firstLine?.amount ?? '—'} €<br />
                        Quantité: {firstLine?.quantity ?? '—'}
                    </>
                );
            })()}
        </div>
        {/* Informations facture */}
        <div className="w-1/3 text-xs">
            {(() => {
                const invoiceDate = (displayedInvoice as any).invoice_date || null;
                const dueDate = (displayedInvoice as any).due_date || null;
                return (
                    <>
                        <strong>Facture:</strong><br />
                        Date: {invoiceDate ? dateToStringFr(new Date(invoiceDate)) : '—'}<br />
                        Deadline: {dueDate ? dateToStringFr(new Date(dueDate)) : '—'}<br />
                        Brouillon: {displayedInvoice.draft ? 'Oui' : 'Non'}
                    </>
                );
            })()}
        </div>
    </div>
</div>
}
```

Affichage corrigé du modal d'import de facture : les informations client, projet et dates utilisent désormais des vérifications sûres (invoice_lines, invoice_date, due_date) pour éviter les erreurs TypeScript. Les champs manquants s'affichent par défaut avec "—", garantissant un rendu stable même si certaines données sont absentes. Le bouton "Rattacher" ne s'active que lorsque facture et utilisateur sont sélectionnés, et l'envoi vers le backend se fait correctement avec création des lignes associées.

Il faut que j'essaie de récupérer les données de la facture pour qu'il puisse s'afficher mais j'ai rencontré cette erreur :

```
⊗ Failed to load resource: the server responded with a status of 404 (Not Found)
⊗ ▶ SyntaxError: Unexpected end of JSON input
⊗ ▶ Erreur lors de la création de la facture : SyntaxError: Unexpected end of JSON input
⊗ Failed to load resource: the server responded with a status of 404 (Not Found)
⊗ ▶ SyntaxError: Unexpected end of JSON input
⊗ ▶ Erreur lors de la création de la facture : SyntaxError: Unexpected end of JSON input
⊗ Failed to load resource: the server responded with a status of 404 (Not Found)
⊗ ▶ SyntaxError: Unexpected end of JSON input
⊗ ▼ Erreur lors de la création de la facture : SyntaxError: Unexpected end of JSON input
    createInvoice @ ModalImportInvoice.tsx:227
```

et ya bien des informations qui sont trouvés mais ne sont pas renvoyées :

DevTools - lucca2penny.localhost/lucca-timeentries

Éléments | Console | Sources | Réseau | Performances | Mémoire | Appli | Confidentialité et sécurité | Lighthouse | Enregistreur

Conserver le journal | Désactiver le cache | Aucune limitation

Filtrer | Inverser | Autres filtres | Tout | Fetch/XHR | Document | CSS | JS | Police | Image | Multimédia | Fichier manifeste | Socket

20 000 ms | 40 000 ms | 60 000 ms | 80 000 ms | 100 000 ms | 120 000 ms | 140 000 ms | 160 000 ms | 180 000 ms | 200 000 ms

Nom

- user-session-status
- departments?fields=id,name,pare...
- invoices?month=2026-01
- users?fields=id,firstName,lastNam...
- timeentries?fields=id,timesheet[id...
- users?fields=id,firstName,lastNam...
- customer_invoices?per_page=500...
- 4080621713
- lucca2penny.localhost

En-têtes | Aperçu | Réponse | Initiateur | Délai | Cookies


```
1  {
-      "id": 4080621713,
-      "label": "Facture SOCIETE OUEST FRANCE - F-2026-01-72 (label généré)",
-      "invoice_number": "F-2026-01-72",
-      "currency": "EUR",
-      "amount": "12000.0",
-      "currency_amount": "12000.0",
-      "currency_amount_before_tax": "10000.0",
-      "tax": "2000.0",
-      "currency_tax": "2000.0",
-      "remaining_amount_with_tax": "12000.0",
-      "remaining_amount_without_tax": "10000.0",
-      "draft": false,
-      "language": "fr_FR",
-      "paid": false,
-      "status": "upcoming",
-      "filename": "Facture-SOCIETE OUEST FRANCE-Sandbox Comptabilité d'engagement - thomas@delia.tech-F-2026-01-72.pdf",
-      "date": "2026-01-30",
-      "deadline": "2026-03-01",
-      "special_mention": null,
```


9 requête(s) sur 192 | 124 ko/6 153 ko

- Logs avec Docker également :

lucca-2-penny-backend-1

9866a7de2664 | lucca-2-penny-backend:latest

STATUS
Running (1 second ago)

Logs | Inspect | Bind mounts | Exec | Files | Stats

```
2026-01-30T13:25:59.983Z info: Received a GET request for /api/lucca/users?
fields=id,firstName,lastName,department[id,name],applicationData[profile_figgo]&orderBy=firstName,asc,lastName,asc
[HPM] ECONNRESET: Error: socket hang up
    at TLSSocket.socketOnEnd (node:_http_client:528:25)
    at TLSSocket.emit (node:events:536:35)
    at endReadableNT (node:internal/streams/readable:1698:12)
    at processTicksAndRejections (node:internal/process/task_queues:82:21) {
  code: 'ECONNRESET'
}
2026-01-30T13:26:18.117Z info: Received a GET request for /api/pennylane/customer_invoices?per_page=500&filter=
[%7B%22field%22:%22id%22,%22operator%22:%22eq%22,%22value%22:%22F-2026-01-73%22%7D]
2026-01-30T13:26:27.806Z info: Received a GET request for /api/lucca/users?
fields=id,firstName,lastName,department[id,name],applicationData[profile_figgo]&orderBy=firstName,asc,lastName,asc
```

Problème : F-2026-01-73 n'est PAS un id Pennylane, c'est un invoice_number (numéro humain), pas l'ID technique. Dans Pennylane : Donc ta requête revient forcément vide → null.

- Puis j'ai testé d'afficher en ajoutant une métode de recherche par id et dans la console on trouve :

```
Recherche facture par numéro:                                   ModalImportInvoice.tsx:92
Facture récupérée depuis Pennylane:                             ModalImportInvoice.tsx:99
  ▸ {id: 4089616763, label: 'Facture DEVOTEAM (label généré)', invoice_number: '', currency: 'EUR', amount: '252.0', …}
Recherche facture par numéro: F-2026-01-73                      ModalImportInvoice.tsx:92
Facture récupérée depuis Pennylane:                             ModalImportInvoice.tsx:99
  ▸ {id: 4080631669, label: 'Facture DEVOTEAM - F-2026-01-73 (label généré)', invoice_number: 'F-2026-01-73', currency: 'EU
    R', amount: '252.0', …}
>
```

Pour F-2026-01-73, invoice_number est bien présent ("F-2026-01-73") → la facture est récupérée correctement. Donc le problème n'est pas dans ton code, c'est dans les données Pennylane : certaines factures ont un invoice_number vide, d'où le null lors de la recherche par numéro.
Pusi j'ai aj modifié pour ne plus avoir cette erreur, fait afficher l'objet par des logs :

```
Recherche facture par numéro:                                          ModalImportInvoice.tsx:92
Facture récupérée depuis Pennylane:                                    ModalImportInvoice.tsx:99
 ▸ {id: 4089616763, label: 'Facture DEVOTEAM (label généré)', invoice_number: '', currency: 'EUR', amount: '252.0', …}
Recherche facture par numéro: F-2026-01-73                             ModalImportInvoice.tsx:92
Facture récupérée depuis Pennylane:                                    ModalImportInvoice.tsx:99
 ▾ {id: 4080631669, label: 'Facture DEVOTEAM - F-2026-01-73 (label généré)', invoice_number: 'F-2026-01-73', currency: 'EU
   R', amount: '252.0', …} ℹ
     amount: "252.0"
   ▸ appendices: {url: 'https://app.pennylane.com/api/external/v2/customer_invoices/4080631669/appendices'}
     archived_at: null
     billing_subscription: null
   ▸ categories: {url: 'https://app.pennylane.com/api/external/v2/customer_invoices/4080631669/categories'}
     created_at: "2026-01-30T08:30:36.129078Z"
     credited_invoice: null
     currency: "EUR"
     currency_amount: "252.0"
     currency_amount_before_tax: "210.0"
     currency_tax: "42.0"
   ▸ custom_header_fields: {url: 'https://app.pennylane.com/api/external/v2/customer_invoices/4080631669/custom_header_fie
   ▾ customer:
       id: 118098334
       url: "https://app.pennylane.com/api/external/v2/customers/118098334"
     ▾ [[Prototype]]: Object
       ▸ constructor: ƒ Object()
       ▸ hasOwnProperty: ƒ hasOwnProperty()
       ▸ isPrototypeOf: ƒ isPrototypeOf()
       ▸ propertyIsEnumerable: ƒ propertyIsEnumerable()
       ▸ toLocaleString: ƒ toLocaleString()
       ▸ toString: ƒ toString()
       ▸ valueOf: ƒ valueOf()
       ▸ __defineGetter__: ƒ __defineGetter__()
       ▸ __defineSetter__: ƒ __defineSetter__()
       ▸ __lookupGetter__: ƒ __lookupGetter__()
       ▸ __lookupSetter__: ƒ __lookupSetter__()
         __proto__: (...)
       ▸ get __proto__: ƒ __proto__()
       ▸ set __proto__: ƒ __proto__()
   ▾ customer_invoice_template:
       id: 124422
     ▸ [[Prototype]]: Object
     date: "2026-01-30"
     deadline: "2026-03-01"
   ▸ discount: {type: 'relative', value: '0.0'}
```

Puis j'ai eu un problème car il m'indiquait qu'il n'avait aucune ligne de factures :

puis j'ai modifié la modale qui va permettre de récupérer les informations de la facture avant de la rattacher :

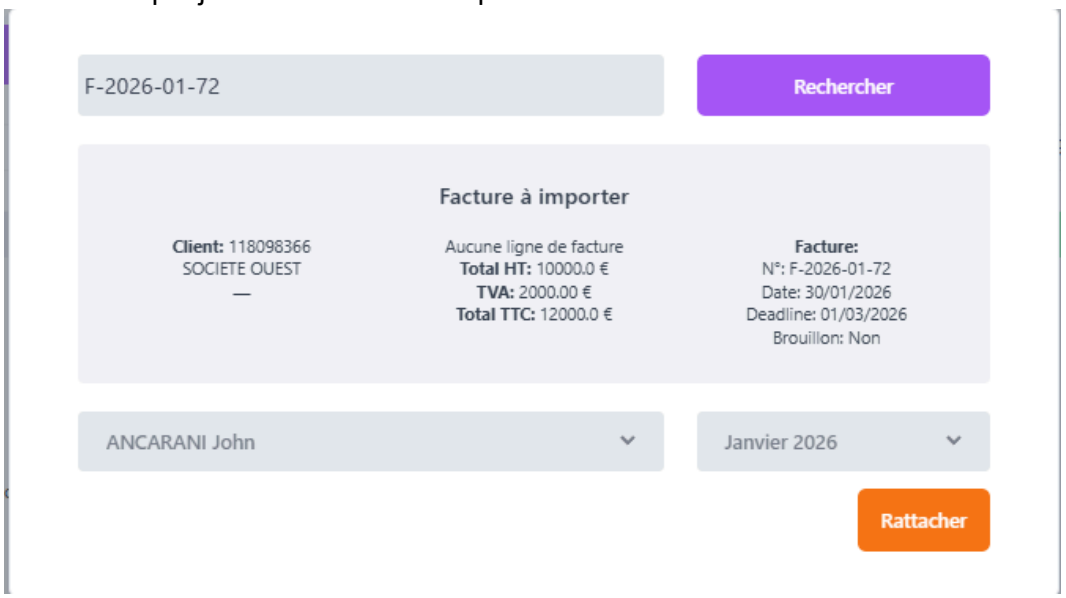
```
{displayedInvoice && (
    <div className="flex flex-col py-6 px-6 mt-5 space-y-4 bg-gray-100 rounded">
        <h2 className="text-md font-semibold text-center">Facture à importer</h2>
        <div className="flex space-x-6 mt-2">

        {/* Client */}
        <div className="w-1/3 text-xs">
        <strong>Client:</strong> {displayedInvoice.customer?.id || '—'}<br />
        <span>{getClientName(displayedInvoice)}</span><br />
        {displayedInvoice.customer?.billing_address?.postal_code || '—'}<br />
        {displayedInvoice.customer?.billing_address?.city || '—'}<br />
        </div>

        {/* Première ligne de facture */}
        <div className="w-1/3 text-xs">
        {displayedInvoice.invoice_lines?.length ? (
            <>
            <strong>Projet:</strong> {displayedInvoice.invoice_lines[0].label}<br />
            Prix HT: {displayedInvoice.invoice_lines[0].currency_price_before_tax} €<br />
            Prix TTC: {displayedInvoice.invoice_lines[0].amount} €<br />
            Quantité: {displayedInvoice.invoice_lines[0].quantity}
            </>
        ) : (
            <>
            <div>Aucune ligne de facture</div>
            <div>
                <strong>Total HT:</strong> {displayedInvoice.currency_amount_before_tax} €<br />
                <strong>TVA:</strong> {(
                    Number(displayedInvoice.currency_amount) -
                    Number(displayedInvoice.currency_amount_before_tax)
                ).toFixed(2)} €<br />
                <strong>Total TTC:</strong> {displayedInvoice.currency_amount} €
            </div>
            </>
        )}
        </div>

        {/* Infos facture */}
        <div className="w-1/3 text-xs">
            <strong>Facture:</strong><br />
            N°: {displayedInvoice.invoice_number || '—'}<br />
            Date: {formatDate(displayedInvoice.date ?? displayedInvoice.invoice_date)}<br />
            Deadline: {formatDate(displayedInvoice.deadline ?? displayedInvoice.due_date)}<br />
            Brouillon: {displayedInvoice.draft ? 'Oui' : 'Non'}
        </div>
        </div>
    </div>
```

Et voici ce que j'ai réussi à rattacher pour l'instant :



Observations éventuelles :

La génération des factures et la gestion des statuts ont été très complexes à corriger cette semaine. Le passage du symbole euro en orange puis en vert a nécessité l'analyse de la logique de la nouvelle API Pennylane (migration de certaines variables, nouveaux attendues dans l'affichage, …), des logs et la mise à jour du code front et back. Le processus fonctionne désormais correctement, mais certains ajustements restent à surveiller pour garantir la mise à jour automatique dans tous les cas d'utilisation.

Objectifs pour la semaine suivante *(obligatoire)* :

- Récupérer les autres informations côté client pour importer une facture et finir le rattachement de facture (ville et le code postal que j'ai pas encore réussi à récupérer)
- Faire fonctionner le bouton Rattacher